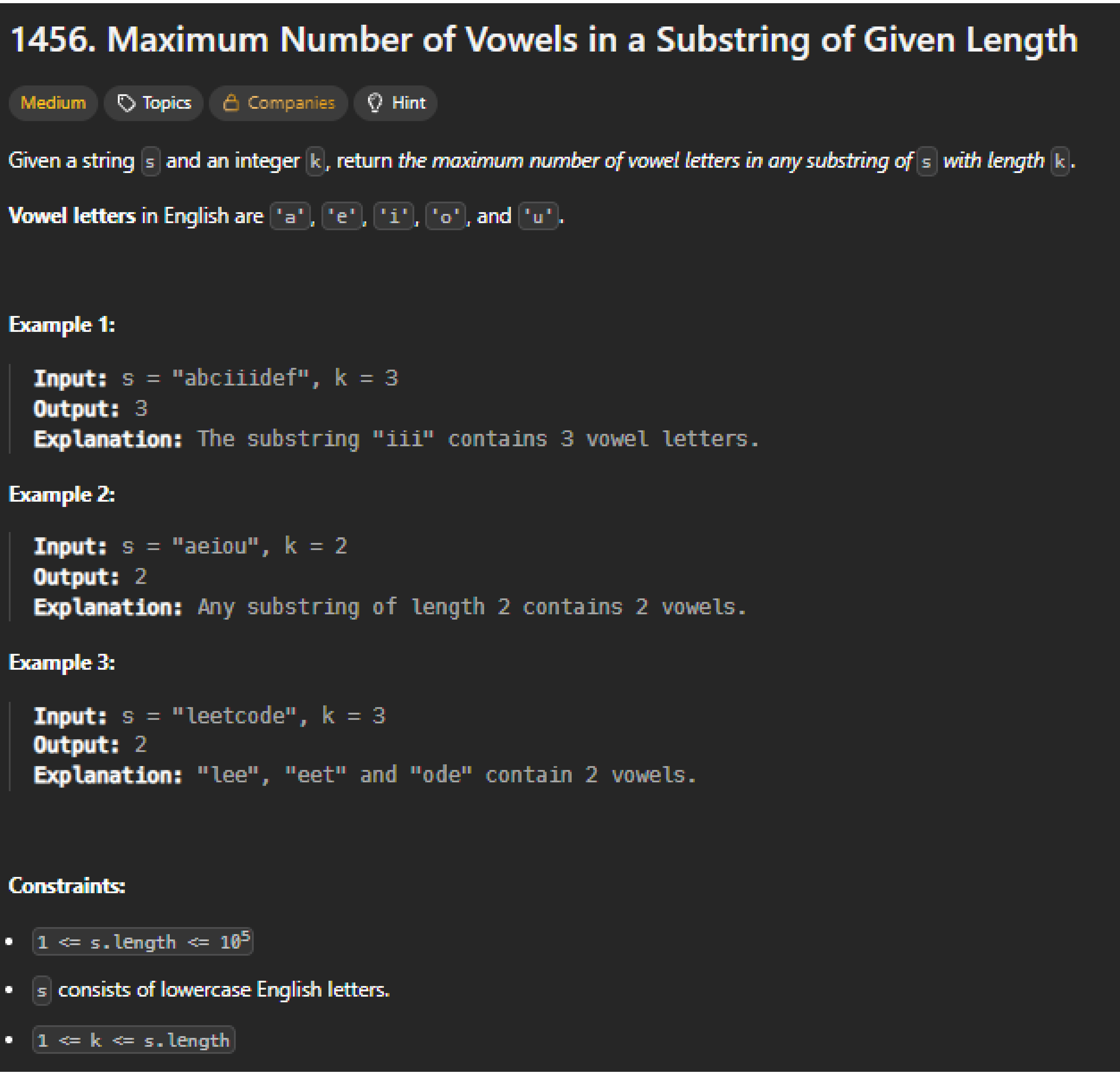


1456. Maximum Number of Vowels in a Substring of Given Length

Given a string `s` and an integer `k`, return *the maximum number of vowel letters in any substring of* `s` *with length* `k`.

Vowel letters in English are `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`.

Example 1:

> **Input:** s = "abciiidef", k = 3
> **Output:** 3
> **Explanation:** The substring "iii" contains 3 vowel letters.

Example 2:

> **Input:** s = "aeiou", k = 2
> **Output:** 2
> **Explanation:** Any substring of length 2 contains 2 vowels.

Example 3:

> **Input:** s = "leetcode", k = 3
> **Output:** 2
> **Explanation:** "lee", "eet" and "ode" contain 2 vowels.

Constraints:

- $1 <= s.length <= 10^5$
- `s` consists of lowercase English letters.
- `1 <= k <= s.length`

DS/Day-88/4-11-26

Leetcode - 1456, Maximum Number of Vowels in a substring of Given Length...

this is a sliding window problem.

You're given : → a string s
→ an integer k.

you need to find the max. no. of vowels (a, e, i, o, u) present in any substring of length exactly k.

example→ s = "abciiidef"
k = 3

possible substring of length 3,

substring	vowels
abc	1
bci	1
cii	2
iii	3
iid	2
ide	2
def	1

answer = 3, bcuz "iii" contains 3 vowels.

fixed-size sliding window approach→

instead of checking every substring from scratch, we maintain a window of size k. so, instead of rescanning all k chars, we → 1) remove the contribution of the outgoing char.
2) add the " " " incoming ".

imagine:-

s = $\underbrace{a\ b\ c}_{k=3.}$ i i i d e f

s = a$\underbrace{bci}$iidef
→ a went out of window
→ i entered the window
→ b/c remained inside

↓ only if the outgoing char is a vowel.

↑ only if the incoming char is a vowel

when we move the window one position to the right:
→ remove the char leaving the window
→ add the new char entering the window
→ update the vowel count.

this avoids recounting and repeating all chars.

class Solution { public:
int maxVowels(string s, int k) {

int n = s.length(); → stores the length of the string
here, n = 9.

int count = 0; → no. of vowels in the current window.
~~int maxi = 0;~~
~~int count~~
int maxCount = 0; → maximum no. of vowels found in any window so far.

// count vowels in the 1st window

for (int i = 0; i < k; i++) { → before sliding the window, we must calc. the vowel count of 1st window, since k = 3, the loop checks, s[0], s[1], s[2].

if (s[i] == 'a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'o' || s[i] == 'u') { → checks whether the char is a vowel.

count ++; → ↑ the vowel count if the character is a vowel. after checking "abc": count = 1.

} }

maxCount = count; our 1st window has one vowel, therefore our window maximum is also one.

// slide the window, sliding window loop.

for(int i = k; i<n ;i++){ why start from k? bcz indices 0 to k-1 were already used for the 1st window, next char to enter is at index k.

↓ i=k means the 1st incoming char is at index k, here, 3, i<n means we continue until the last valid index, which is n-1 = 8,

if (s[i-k]== 'a' || s[i-k] == 'e' || s[i-k] == 'i' || s[i-k] == 'o' || s[i-k] == 'u')

if a vowel, ↓ the count. ← { count--; remove the char leaving the window.

} i-k? it represents the index of the char that is leaving the current window. when i=3, i-k = 3-3=0, s[0]=a leaving the window.

if (s[i] == 'a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'o' || s[i] == 'u'){ add the char entering the window.

for i=3, s[3]=e; if a vowel, ↑ the count.

count ++;

}

// update maximum vowel count.

maxCount = max(maxCount, count);

}

return maxCount;

}

};

TC = O(n) SC = O(1)

↳ 1st loop checks the 1st k chars.

↳ 2nd loop visits the remaining n-k chars,

↳ each movement takes const. time, O(1), bcz we only check two chars. O(k)+O(n-k) = O(n)

complete dryrun

s = "abciiidef"
k = 3.

__/__/____

initial window:

a b c i i i d e f
0 1 2, 3 4 5 6 7 8.

window = abc | vowel count = 1.

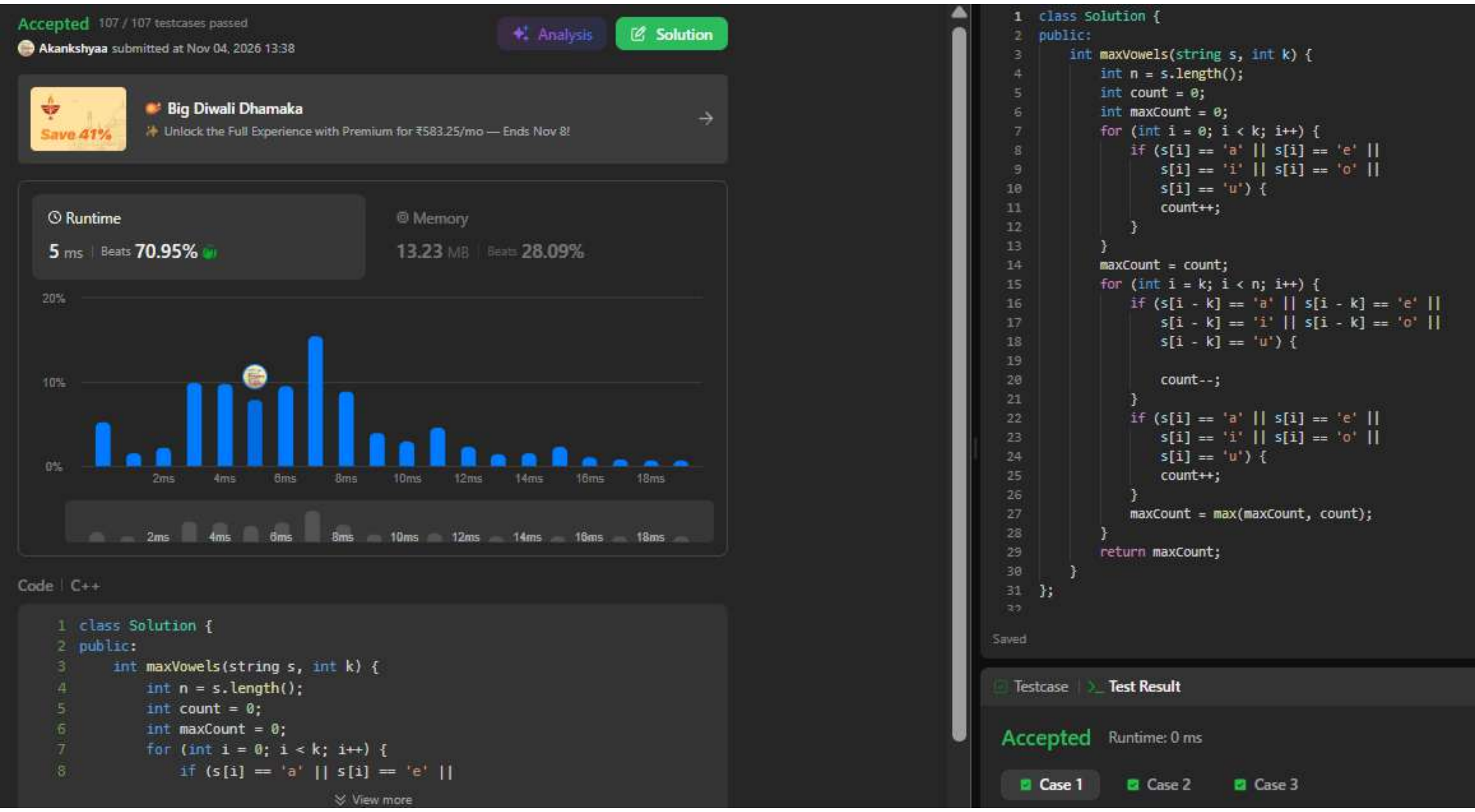
iteration	outgoing	incoming	newwindow	count	maximum
initial	–	–	abc	1	1
1	a	i	bci	1	1
2	b	i	cii	2	2
3	c	i	iii	3	3
4	i	d	iid	2	3
5	i	e	ide	2	3
6	i	f	def	1	3

iter 1; i = 3, outgoing char, s[i-k] = s[3-3] = s[0] = 'a'; a is a vowel, count = 1-1 = 0,
incoming char, s[i] = s[3] = 'i'; i is a vowel, count = 0+1 = 1.

iter 2, i = 4, outgoing char, s[i-k] = s[4-3] = s[1] = 'b'; not a vowel,
incoming char, s[4] = 'i'. is vowel, count = 2,
new window = "cii".

iter 3, i = 5, outgoing = c, incoming i
s, [illegible] count = 3,
new window = "iii".
Now, maxcount = max (2, 3) ; = 3,

So, 3, "iii"
↳ the winning substring.



Accepted 107 / 107 testcases passed

Akankshyaa submitted at Nov 04, 2026 13:38

Save 41%

Big Diwali Dhamaka

Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

5 ms | Beats 70.95%

Memory

13.23 MB | Beats 28.09%

Code | C++

```cpp
class Solution {
public:
    int maxVowels(string s, int k) {
        int n = s.length();
        int count = 0;
        int maxCount = 0;
        for (int i = 0; i < k; i++) {
            if (s[i] == 'a' || s[i] == 'e' ||
```

```cpp
class Solution {
public:
    int maxVowels(string s, int k) {
        int n = s.length();
        int count = 0;
        int maxCount = 0;
        for (int i = 0; i < k; i++) {
            if (s[i] == 'a' || s[i] == 'e' ||
                s[i] == 'i' || s[i] == 'o' ||
                s[i] == 'u') {
                count++;
            }
        }
        maxCount = count;
        for (int i = k; i < n; i++) {
            if (s[i - k] == 'a' || s[i - k] == 'e' ||
                s[i - k] == 'i' || s[i - k] == 'o' ||
                s[i - k] == 'u') {

                count--;
            }
            if (s[i] == 'a' || s[i] == 'e' ||
                s[i] == 'i' || s[i] == 'o' ||
                s[i] == 'u') {
                count++;
            }
            maxCount = max(maxCount, count);
        }
        return maxCount;
    }
};
```

Accepted Runtime: 0 ms

☑ Case 1 ☑ Case 2 ☑ Case 3